

早期returnでコードをきれいに

ネストを減らして読みやすいコードを書こう

早期returnとは？

条件に合わない場合は早めに処理を終了すること

ネストが深い例（読みにくい）

```
function processUser(user) {  
  if (user) {  
    if (user.isActive) {  
      if (user.age >= 18) {  
        if (user.hasPermission) {  
          // メインの処理  
          console.log('ユーザー処理を実行');  
          return user.name;  
        } else {  
          return '権限がありません';  
        }  
      } else {  
        return '18歳未満です';  
      }  
    } else {  
      return 'アカウントが無効です';  
    }  
  } else {  
    return 'ユーザーが存在しません';  
  }  
}
```

早期returnで改善（読みやすい）

```
function processUser(user) {  
  // 早期return - 条件に合わない場合はすぐに終了  
  if (!user) return 'ユーザーが存在しません';  
  if (!user.isActive) return 'アカウントが無効です';  
  if (user.age < 18) return '18歳未満です';  
  if (!user.hasPermission) return '権限がありません';  
  
  // メインの処理 - ネストなし！  
  console.log('ユーザー処理を実行');  
  return user.name;  
}
```

Reactコンポーネントでの例

ネストが深い例

```
function UserProfile({ user, loading, error }) {  
  return (  
    <div>  
      {!loading ? (  
        !error ? (  
          user ? (  
            user.isVerified ? (  
              <div>  
                <h1>{user.name}</h1>  
                <p>{user.email}</p>  
              </div>  
            ) : (  
              <div>アカウントが未認証です</div>  
            )  
          ) : (  
            <div>ユーザーが見つかりません</div>  
          )  
        ) : (  
          <div>エラー: {error.message}</div>  
        )  
      ) : (  
        <div>読み込み中...</div>  
      )  
    </div>  
  )  
}
```

早期returnで改善

```
function UserProfile({ user, loading, error }) {  
  // 早期return  
  if (loading) return <div>読み込み中...</div>;  
  if (error) return <div>エラー: {error.message}</div>;  
  if (!user) return <div>ユーザーが見つかりません</div>;  
  if (!user.isVerified) return <div>アカウントが未認証です</div>;  
  
  // メインのレンダリング - ネストなし!  
  return (  
    <div>  
      <h1>{user.name}</h1>  
      <p>{user.email}</p>  
    </div>  
  );  
}
```

CRUDとは？

地図アプリのマーカー情報を管理するAPIの基本操作

CRUD = 4つの基本操作

- **Create**（作成） - 新しいマーカーを作成
- **Read**（読み取り） - マーカー情報を取得
- **Update**（更新） - マーカーの情報を更新
- **Delete**（削除） - マーカーを削除

REST APIとHTTPメソッドの対応

操作	HTTPメソッド	エンドポイント	説明
Create	POST	/markers	新規マーカー作成
Read	GET	/markers	全マーカー取得

操作	HTTPメソッド	エンドポイント	説明
Read	GET	/markers/:id	特定マーカー取得
Read	GET	/markers? lat=35&lng=139&radius=1000	範囲検索
Update	PUT	/markers/:id	マーカー情報更新
Delete	DELETE	/markers/:id	マーカー削除

axiosでの実装イメージ

```
// 例：新しい店舗マーカーを作成
const newMarker = {
  name: '新宿店',
  lat: 35.689,
  lng: 139.691,
  category: 'convenience',
};
```

```
// APIにPOSTリクエストを送信
const response = await axios.post('/api/markers', newMarker);
```

非同期処理の理解

同期処理と非同期処理の違いを理解しよう

同期処理と非同期処理の違い

同期処理 = すぐに結果が返る

```
const price = 1000;  
const tax = price * 0.1;  
console.log(tax); // 100がすぐに表示される
```

非同期処理 = 結果が返るまで時間がかかる

```
// APIからマーカー情報を取得（ネットワーク通信が必要）  
const response = axios.get('/api/markers/1');  
console.log(response); // Promise {<pending>} と表示される
```

Promiseとは

「あとで結果を渡します」という約束のオブジェクト

```
axios
  .get('/api/markers/1')
  .then((response) => {
    // 成功した時の処理
    console.log('マーカー情報:', response.data);
  })
  .catch((error) => {
    // 失敗した時の処理
    console.error('エラー発生:', error);
  });
```

async/awaitでもっと読みやすく

従来のPromise (then/catch)

```
function getMarkerInfo() {  
  axios  
    .get('/api/markers/1')  
    .then((response) => {  
      console.log(response.data);  
    })  
    .catch((error) => {  
      console.error(error);  
    });  
}
```

async/awaitを使った書き方

```
async function getMarkerInfo() {  
  try {  
    const response = await axios.get('/api/markers/1');  
    console.log(response.data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```


Promise.allで複数のAPIを同時に呼ぶ

逐次実行（遅い）

```
async function getMarkersSequential() {  
  const tokyo = await axios.get('/api/markers?area=tokyo');  
  const osaka = await axios.get('/api/markers?area=osaka');  
  const fukuoka = await axios.get('/api/markers?area=fukuoka');  
  // 各APIを順番に待つので時間がかかる  
}
```

並行実行 (速い)

```
async function getMarkersParallel() {  
  const [tokyo, osaka, fukuoka] = await Promise.all([  
    axios.get('/api/markers?area=tokyo'),  
    axios.get('/api/markers?area=osaka'),  
    axios.get('/api/markers?area=fukuoka'),  
  ]);  
  // 3つのAPIを同時に呼ぶので速い  
}
```

TypeScriptの活用

ジェネリクスで型安全なコードを書こう

ジェネリクスとは？

「型の変数」のようなもの - 同じ処理で違う型を扱えるようにする仕組み

```
// ジェネリクスを使わない場合
interface MarkerResponse {
    data: Marker;
}

interface UserResponse {
    data: User;
}

interface StoreResponse {
    data: Store;
}
// 型ごとに似たようなインターフェースを作る必要がある...
```

ジェネリクスで型を再利用

```
// ジェネリクスを使った場合
interface ApiResponse<T> {
    data: T;
    status: number;
    message: string;
}

// 使い回せる！
type MarkerResponse = ApiResponse<Marker>;
type UserResponse = ApiResponse<User>;
type StoreResponse = ApiResponse<Store>;
```

axiosでの活用例

```
// マーカーの型定義
interface Marker {
  id: number;
  name: string;
  lat: number;
  lng: number;
}
```

```
// ジェネリクスでレスポンスの型を指定
async function getMarker(id: number) {
  const response = await axios.get<Marker>(`/api/markers/${id}`);
  // response.dataの型がMarkerになる！
  console.log(response.data.name); // 型補完が効く
}
```

```
// 配列の場合
async function getAllMarkers() {
  const response = await axios.get<Marker[]>('/api/markers');
  response.data.forEach((marker) => {
    console.log(marker.lat, marker.lng); // 型が分かっているので安全
  });
}
```


実践的な使い方

```
// APIクライアントの共通化
class ApiClient {
  async get<T>(url: string): Promise<T> {
    const response = await axios.get<T>(url);
    return response.data;
  }
}

// 使用例
const client = new ApiClient();
const marker = await client.get<Marker>('/api/markers/1');
const stores = await client.get<Store[]>('/api/stores');
```

axiosのカプセル化

API呼び出しを共通化して保守性を向上させよう

カプセル化とは？

複雑な処理を隠して、シンプルなインターフェースを提供すること

カプセル化前（各コンポーネントで直接axios）

```
// UserList.jsx
const users = await axios.get('/api/users');

// ProductList.jsx
const products = await axios.get('/api/products');

// OrderHistory.jsx
const orders = await axios.get('/api/orders');
```

問題点

- 同じような処理が散らばる
- エラーハンドリングが統一されない
- APIのURL変更時に全箇所を修正が必要

APIクライアントの作成

```
// api/client.js
import axios from 'axios';

const BASE_URL = 'http://localhost:3000/api';
const TIME_OUT = 1000;

// 共通の設定を持つaxiosインスタンスを作成
const apiClient = axios.create({
  baseURL: BASE_URL,
  timeout: TIME_OUT,
  headers: {
    'Content-Type': 'application/json',
  },
});

export default apiClient;
```

カプセル化後のコンポーネント

```
// components/MarkerList.jsx
import apiClient from '../api/client';

function MarkerList() {
  const [markers, setMarkers] = useState([]);

  useEffect(() => {
    const fetchMarkers = async () => {
      try {
        // 共通設定が適用されたaxiosを使用
        const response = await apiClient.get('/markers');
        setMarkers(response.data);
      } catch (error) {
        console.error('マーカー取得エラー:', error);
      }
    };

    fetchMarkers();
  }, []);
}
```

カプセル化のメリット

1. 共通設定の管理

- baseURLやheadersを一箇所で管理
- 設定変更時の修正が簡単

2. コードの再利用

- 同じ設定を複数のコンポーネントで使える
- 重複するコードが削減される

3. 保守性の向上

- APIのURL変更時の修正箇所が限定される
- 設定の一元管理でミスが減る

